

Insertion Sort

Learn how holding one value and shifting others makes room in an ordered section.

How to use this chapter:

Learn the idea in plain language, read each diagram, predict every trace row, then study the commented Java and attempt the exam-style questions.

Start with the idea

Insertion sort grows an ordered section on the left. Save the next value separately, move larger values one position right, then place the saved value in the space they leave.

A small example

Start with [85, 12, 59, 45, 72, 51]. The first value, 85, is already an ordered section on its own.

Save 12 separately. The code calls this held value the **key**. It stays safe while we make room.

In the next illustration, track the held key separately from the values shifting right. The drawn gap shows where the key can go; the actual array can briefly contain a duplicate after a shift.

The whole process at a glance

Hold the next value separately as the key. Copy larger values one position right to make room, then insert the key. The drawn gap helps you follow the opening; the array can briefly contain a duplicate after a shift.

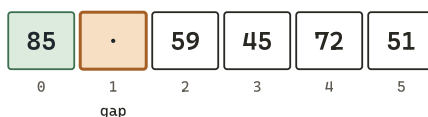
Green: sorted

Blue: being checked

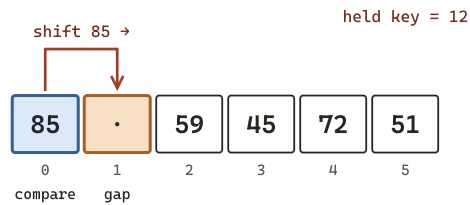
Orange: space for the held value

Pass 1: hold 12, shift, insert

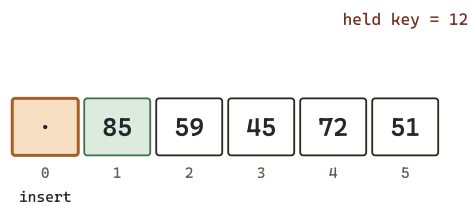
held key = 12



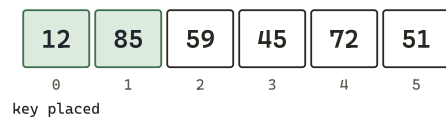
Start with [85, 12, 59, 45, 72, 51]. Hold 12 from index 1 separately. Positions 0 through 0 are already ordered. Compare the held 12 with 85 at index 0. The gap marks where we are making room; it is not an empty Java array cell.



85 > 12 is true, so 85 belongs after 12. Shift 85 from index 0 to index 1 by copying it right. With 12 still held, the diagram row becomes [gap, 85, 59, 45, 72, 51]. There is no value further left; stop shifting.

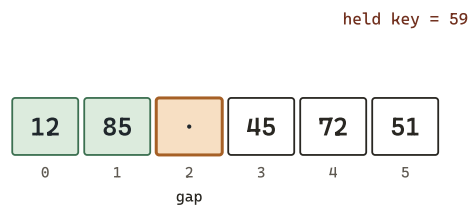


We reached the start of the row: index -1 is outside the array, so stop without reading it. Insert the held 12 at index 0. The row becomes [12, 85, 59, 45, 72, 51].

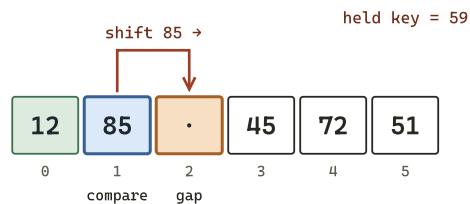


Positions 0 through 1 are now ordered: [12, 85]. Next pass, hold 59 from index 2 and compare it with 85 at index 1.

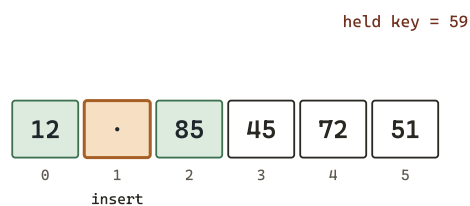
Pass 2: hold 59, shift, insert



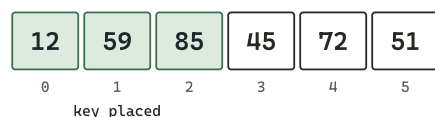
Start with [12, 85, 59, 45, 72, 51]. Hold 59 from index 2 separately. Positions 0 through 1 are already ordered. Compare the held 59 with 85 at index 1. The gap marks where we are making room; it is not an empty Java array cell.



85 > 59 is true, so 85 belongs after 59. Shift 85 from index 1 to index 2 by copying it right. With 59 still held, the diagram row becomes [12, gap, 85, 45, 72, 51]. Next compare 12 at index 0 with the held 59.

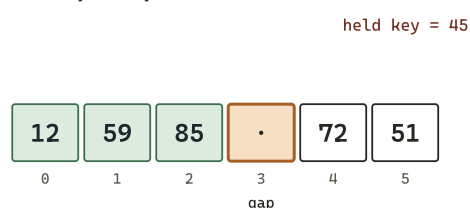


12 > 59 is false, so stop shifting: 59 belongs after 12 at index 0. Insert the held 59 at index 1. The row becomes [12, 59, 85, 45, 72, 51].

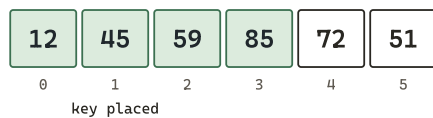
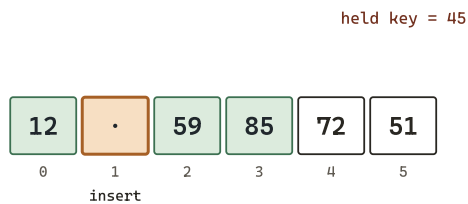
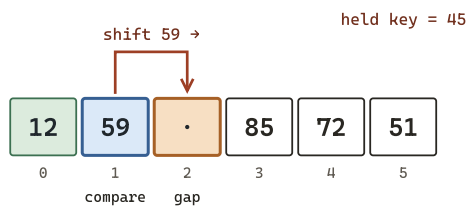
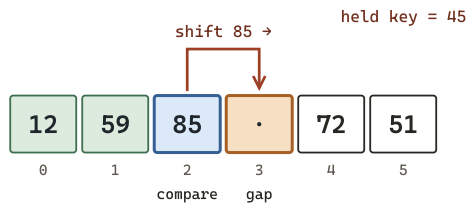


Positions 0 through 2 are now ordered: [12, 59, 85]. Next pass, hold 45 from index 3 and compare it with 85 at index 2.

Pass 3: hold 45, shift, insert



Start with [12, 59, 85, 45, 72, 51]. Hold 45 from index 3 separately. Positions 0 through 2 are already ordered. Compare the held 45 with 85 at index 2. The gap marks where we are making room; it is not an empty Java array cell.



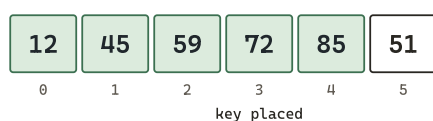
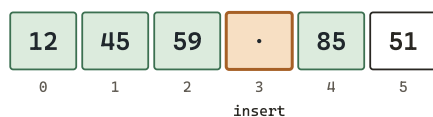
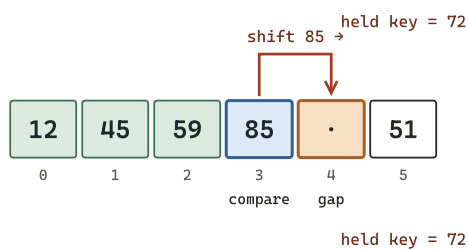
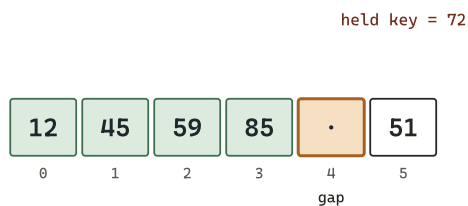
85 > 45 is true, so 85 belongs after 45. Shift 85 from index 2 to index 3 by copying it right. With 45 still held, the diagram row becomes [12, 59, gap, 85, 72, 51]. Next compare 59 at index 1 with the held 45.

59 > 45 is true, so 59 belongs after 45. Shift 59 from index 1 to index 2 by copying it right. With 45 still held, the diagram row becomes [12, gap, 59, 85, 72, 51]. Next compare 12 at index 0 with the held 45.

12 > 45 is false, so stop shifting: 45 belongs after 12 at index 0. Insert the held 45 at index 1. The row becomes [12, 45, 59, 85, 72, 51].

Positions 0 through 3 are now ordered: [12, 45, 59, 85]. Next pass, hold 72 from index 4 and compare it with 85 at index 3.

Pass 4: hold 72, shift, insert



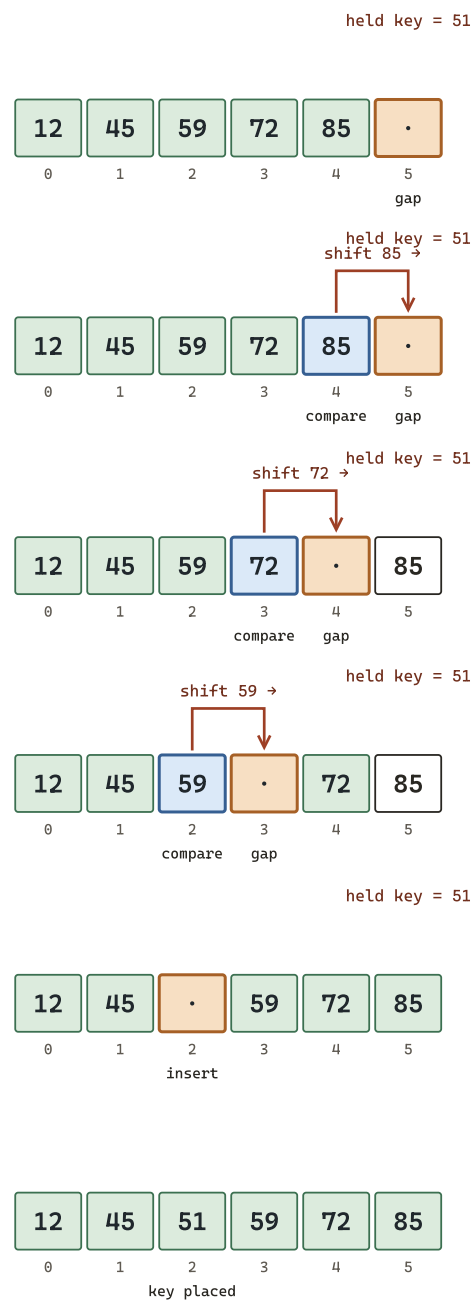
Start with [12, 45, 59, 85, 72, 51]. Hold 72 from index 4 separately. Positions 0 through 3 are already ordered. Compare the held 72 with 85 at index 3. The gap marks where we are making room; it is not an empty Java array cell.

85 > 72 is true, so 85 belongs after 72. Shift 85 from index 3 to index 4 by copying it right. With 72 still held, the diagram row becomes [12, 45, 59, gap, 85, 51]. Next compare 59 at index 2 with the held 72.

59 > 72 is false, so stop shifting: 72 belongs after 59 at index 2. Insert the held 72 at index 3. The row becomes [12, 45, 59, 72, 85, 51].

Positions 0 through 4 are now ordered: [12, 45, 59, 72, 85]. Next pass, hold 51 from index 5 and compare it with 85 at index 4.

Pass 5: hold 51, shift, insert



Start with [12, 45, 59, 72, 85, 51]. Hold 51 from index 5 separately. Positions 0 through 4 are already ordered. Compare the held 51 with 85 at index 4. The gap marks where we are making room; it is not an empty Java array cell.

85 > 51 is true, so 85 belongs after 51. Shift 85 from index 4 to index 5 by copying it right. With 51 still held, the diagram row becomes [12, 45, 59, 72, gap, 85]. Next compare 72 at index 3 with the held 51.

72 > 51 is true, so 72 belongs after 51. Shift 72 from index 3 to index 4 by copying it right. With 51 still held, the diagram row becomes [12, 45, 59, gap, 72, 85]. Next compare 59 at index 2 with the held 51.

59 > 51 is true, so 59 belongs after 51. Shift 59 from index 2 to index 3 by copying it right. With 51 still held, the diagram row becomes [12, 45, gap, 59, 72, 85]. Next compare 45 at index 1 with the held 51.

45 > 51 is false, so stop shifting: 51 belongs after 45 at index 1. Insert the held 51 at index 2. The row becomes [12, 45, 51, 59, 72, 85].

Positions 0 through 5 are now ordered: [12, 45, 51, 59, 72, 85]. There is no next value to hold; the whole row is sorted, so stop.

Sorted: [12, 45, 51, 59, 72, 85]. Each held value has been inserted into the growing ordered section. Shifts copy values right; they do not swap two values.

The array we will sort

Detailed reference example: this part introduces its own input, shown below. The opening whole-run map and guided walkthrough keep their original input throughout; each later worked example states its new input explicitly.

Sorting / Insertion sort · The array we will sort

The array we will sort

We will use one array from beginning to end, arranging its values in ascending order—from smallest to largest.

STARTING ARRAY



THE RESULT WE WANT



This example is deliberately uneven. It gives us an iteration with one shift, one with no shifts, one with three shifts, and one with two shifts.

Why this works

The left section is already sorted, so moving backward past its larger values finds the right place for the key. Stop after a value that is no larger than the key, or at the start of the row. A shift copies one value right; a swap would exchange two values.

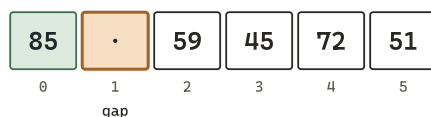
Guided walkthrough: predict the next major step

Predict the next completed pass, then advance. Each scene keeps all its movements together.

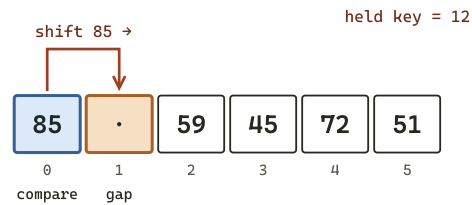
Guided walkthrough: major steps

Pass 1: hold 12, shift, insert

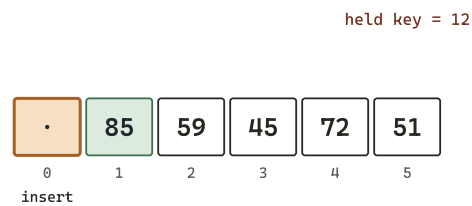
held key = 12



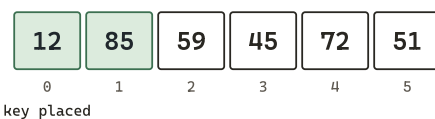
Start with [85, 12, 59, 45, 72, 51]. Hold 12 from index 1 separately. Positions 0 through 0 are already ordered. Compare the held 12 with 85 at index 0. The gap marks where we are making room; it is not an empty Java array cell.



85 > 12 is true, so 85 belongs after 12. Shift 85 from index 0 to index 1 by copying it right. With 12 still held, the diagram row becomes [gap, 85, 59, 45, 72, 51]. There is no value further left; stop shifting.



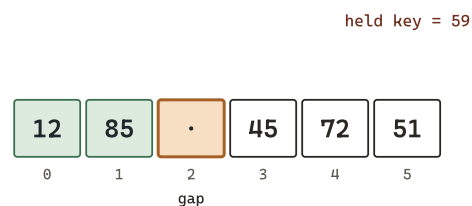
We reached the start of the row: index -1 is outside the array, so stop without reading it. Insert the held 12 at index 0. The row becomes [12, 85, 59, 45, 72, 51].



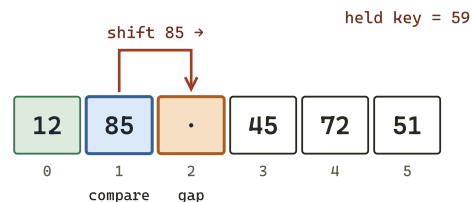
Positions 0 through 1 are now ordered: [12, 85]. Next pass, hold 59 from index 2 and compare it with 85 at index 1.

We reached the start of the row: index -1 is outside the array, so stop without reading it. Insert 12 at index 0.

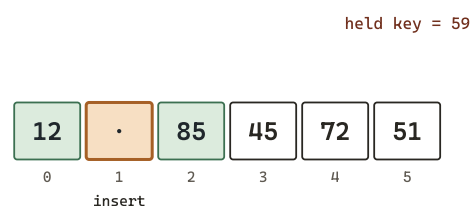
Pass 2: hold 59, shift, insert



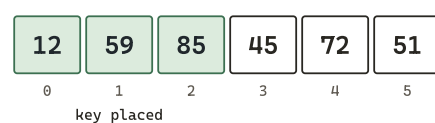
Start with [12, 85, 59, 45, 72, 51]. Hold 59 from index 2 separately. Positions 0 through 1 are already ordered. Compare the held 59 with 85 at index 1. The gap marks where we are making room; it is not an empty Java array cell.



85 > 59 is true, so 85 belongs after 59. Shift 85 from index 1 to index 2 by copying it right. With 59 still held, the diagram row becomes [12, gap, 85, 45, 72, 51]. Next compare 12 at index 0 with the held 59.



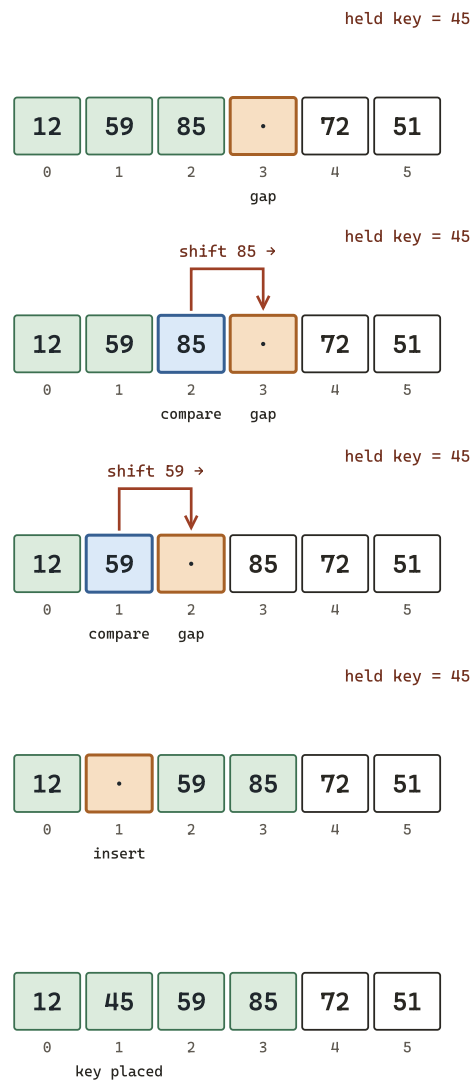
12 > 59 is false, so stop shifting: 59 belongs after 12 at index 0. Insert the held 59 at index 1. The row becomes [12, 59, 85, 45, 72, 51].



Positions 0 through 2 are now ordered: [12, 59, 85]. Next pass, hold 45 from index 3 and compare it with 85 at index 2.

12 > 59 is false, so stop shifting: 59 belongs after 12 at index 0. Insert 59 at index 1.

Pass 3: hold 45, shift, insert



Start with [12, 59, 85, 45, 72, 51]. Hold 45 from index 3 separately. Positions 0 through 2 are already ordered. Compare the held 45 with 85 at index 2. The gap marks where we are making room; it is not an empty Java array cell.

85 > 45 is true, so 85 belongs after 45. Shift 85 from index 2 to index 3 by copying it right. With 45 still held, the diagram row becomes [12, 59, gap, 85, 72, 51]. Next compare 59 at index 1 with the held 45.

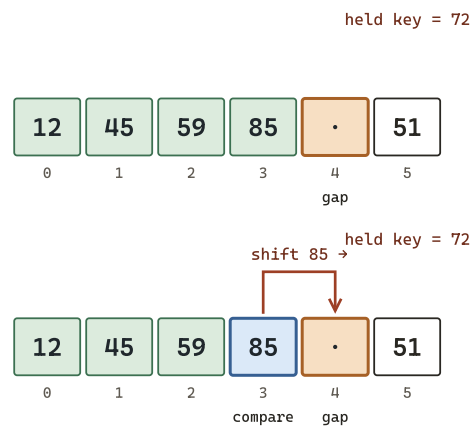
59 > 45 is true, so 59 belongs after 45. Shift 59 from index 1 to index 2 by copying it right. With 45 still held, the diagram row becomes [12, gap, 59, 85, 72, 51]. Next compare 12 at index 0 with the held 45.

12 > 45 is false, so stop shifting: 45 belongs after 12 at index 1. Insert the held 45 at index 1. The row becomes [12, 45, 59, 85, 72, 51].

Positions 0 through 3 are now ordered: [12, 45, 59, 85]. Next pass, hold 72 from index 4 and compare it with 85 at index 3.

12 > 45 is false, so stop shifting: 45 belongs after 12 at index 0. Insert 45 at index 1.

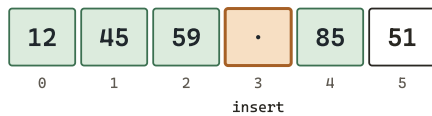
Pass 4: hold 72, shift, insert



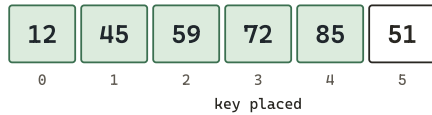
Start with [12, 45, 59, 85, 72, 51]. Hold 72 from index 4 separately. Positions 0 through 3 are already ordered. Compare the held 72 with 85 at index 3. The gap marks where we are making room; it is not an empty Java array cell.

85 > 72 is true, so 85 belongs after 72. Shift 85 from index 3 to index 4 by copying it right. With 72 still held, the diagram row becomes [12, 45, 59, gap, 85, 51]. Next compare 59 at index 2 with the held 72.

held key = 72



59 > 72 is false, so stop shifting: 72 belongs after 59 at index 2. Insert 72 at index 3. The row becomes [12, 45, 59, 72, 85, 51].

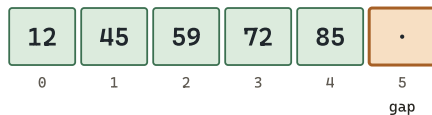


Positions 0 through 4 are now ordered: [12, 45, 59, 72, 85]. Next pass, hold 51 from index 5 and compare it with 85 at index 4.

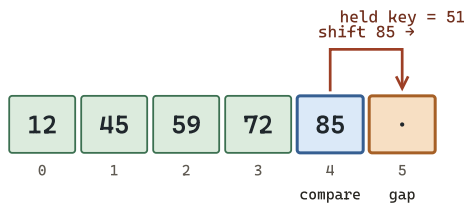
59 > 72 is false, so stop shifting: 72 belongs after 59 at index 2. Insert 72 at index 3.

Pass 5: hold 51, shift, insert

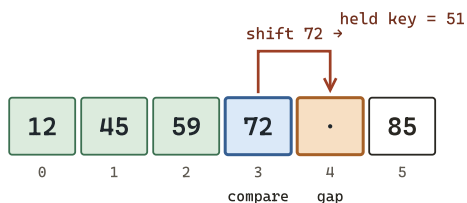
held key = 51



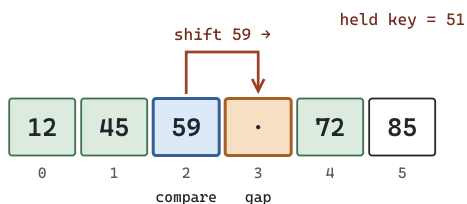
Start with [12, 45, 59, 72, 85, 51]. Hold 51 from index 5 separately. Positions 0 through 4 are already ordered. Compare the held 51 with 85 at index 4. The gap marks where we are making room; it is not an empty Java array cell.



85 > 51 is true, so 85 belongs after 51. Shift 85 from index 4 to index 5 by copying it right. With 51 still held, the diagram row becomes [12, 45, 59, 72, gap, 85]. Next compare 72 at index 3 with the held 51.

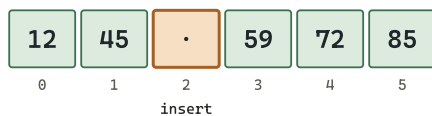


72 > 51 is true, so 72 belongs after 51. Shift 72 from index 3 to index 4 by copying it right. With 51 still held, the diagram row becomes [12, 45, 59, gap, 72, 85]. Next compare 59 at index 2 with the held 51.

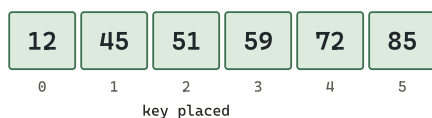


59 > 51 is true, so 59 belongs after 51. Shift 59 from index 2 to index 3 by copying it right. With 51 still held, the diagram row becomes [12, 45, gap, 59, 72, 85]. Next compare 45 at index 1 with the held 51.

held key = 51



45 > 51 is false, so stop shifting: 51 belongs after 45 at index 1. Insert the held 51 at index 2. The row becomes [12, 45, 51, 59, 72, 85].



Positions 0 through 5 are now ordered: [12, 45, 51, 59, 72, 85]. There is no next value to hold; the whole row is sorted, so stop.

45 > 51 is false, so stop shifting: 51 belongs after 45 at index 1. Insert 51 at index 2.

▼ Full trace table - select a row to jump to its scene

PASS	HELD KEY	SHIFTS	INSERT AT	ARRAY AFTER PASS
1	12	1	0	[12, 85, 59, 45, 72, 51]
2	59	1	1	[12, 59, 85, 45, 72, 51]
3	45	2	1	[12, 45, 59, 85, 72, 51]
4	72	1	3	[12, 45, 59, 72, 85, 51]
5	51	3	2	[12, 45, 51, 59, 72, 85]

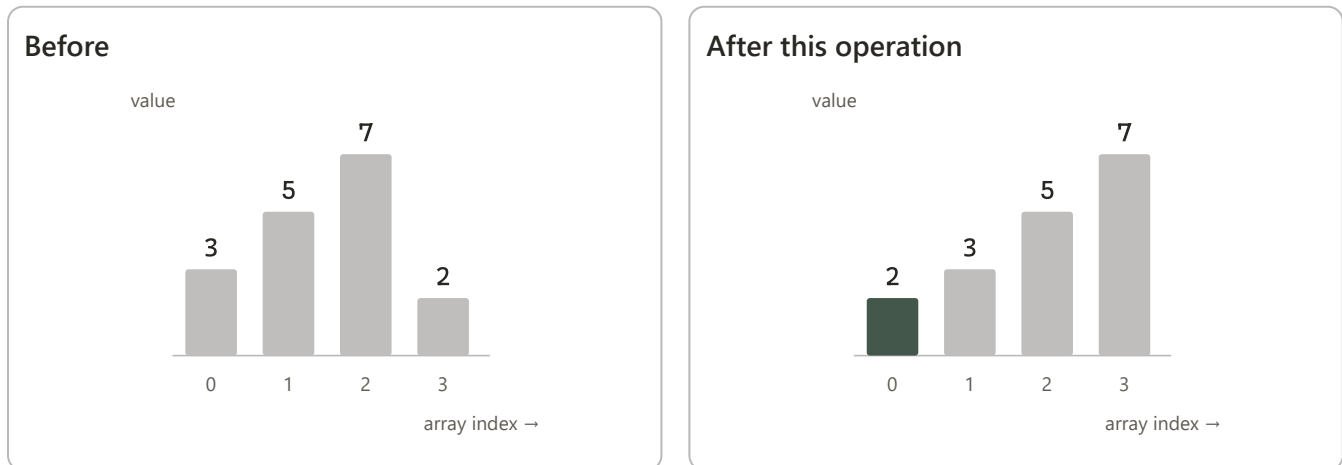
The trace at a glance

PASS	HELD KEY	SHIFTS	INSERT AT	ARRAY AFTER PASS
1	12	1	0	[12, 85, 59, 45, 72, 51]
2	59	1	1	[12, 59, 85, 45, 72, 51]
3	45	2	1	[12, 45, 59, 85, 72, 51]
4	72	1	3	[12, 45, 59, 72, 85, 51]
5	51	3	2	[12, 45, 51, 59, 72, 85]

Read the algorithm as a bar graph

Bar height means **value**; horizontal position means **index**. All panels below use the same height scale. A taller bar is not “more sorted”.

Hold one bar; shift the larger bars right



The first three bars are sorted. Save key 2. Copy 7 one place right, then 5, then 3. Insert held 2 at index 0. This is three shifts plus one insertion, not three swaps. The held copy prevents the value 2 from being lost.

Check yourself: identify which indexes are guaranteed final, which values were merely inspected, and whether the operation swapped, shifted or copied data. A sorted ascending array has non-decreasing heights, including equal-height neighbours.

Fully annotated Java: connect each line to the movement

```
java

public static void insertionSort(int[] arr) {
    // Prefix 0..i-1 is already sorted before each insertion.
    for (int i = 1; i < arr.length; i++) {
        int key = arr[i]; // Preserve this value while its cell is overwritten.
        int j = i - 1;    // Compare backwards through the sorted prefix.
        // Check j first: && avoids evaluating arr[-1].
        // Strict > keeps equal values in their original order (stability).
        while (j ≥ 0 && arr[j] > key) {
            arr[j + 1] = arr[j]; // SHIFT: copy one value one cell right.
            j--;
        }
        // The gap is after the last value ≤ key, or at index 0.
        arr[j + 1] = key;
    }
}
```

Write it from scratch

- Start at the second value.
- Save the key.
- Walk backwards through the sorted prefix.
- Shift strictly larger values right.
- Insert into $j + 1$.

Topic quiz: try it yourself

Answer all 12 questions. Choose an option to see the reasoning; reset to retry. The printable answer key follows the questions.

1. What is the last valid index of an array of length 6?

☐ A. 5

☐ B. 6

☐ C. 7

2. What does $O(1)$ auxiliary space describe?

- ☐ A. Exactly one machine instruction
- ☐ B. The input contains one value
- ☐ C. Extra storage stays bounded as n grows

3. What is a loop invariant?

- ☐ A. A loop that never terminates
- ☐ B. A statement that remains true at a specified loop boundary
- ☐ C. A variable that never changes

4. What does stable sorting preserve?

- ☐ A. The original relative order of records with equal keys
- ☐ B. Every original array index
- ☐ C. The original order of unequal keys

5. What should a correct ascending sort do with `[]` and `[7]`?

- ☐ A. Throw an error for both
- ☐ B. Add a zero to each
- ☐ C. Leave both unchanged

6. Why save `key = arr[i]` before shifting?

- ☐ A. It makes the key an index
- ☐ B. A shift may overwrite its original cell
- ☐ C. It doubles the array length

7. What is a shift?

- ☐ A. Copy one value one position right
- ☐ B. Exchange two values
- ☐ C. Delete a value permanently

8. Where does the held key go when the loop stops?

- ☐ A. `arr[j]`
- ☐ B. `arr[i + 1]`
- ☐ C. `arr[j + 1]`

9. Why check `j >= 0` before `arr[j] > key`?

- ☐ A. Java evaluates the right side first
- ☐ B. Short-circuit evaluation prevents `arr[-1]`
- ☐ C. It prevents duplicate keys

10. Opening map: how many shifts insert the final key 51?

- ☐ A. 3
- ☐ B. 1
- ☐ C. 4

11. Why use `>` rather than `>=` for a stable ascending sort?

- ☐ A. It removes duplicate values
- ☐ B. It makes reverse input linear
- ☐ C. Equal values are not shifted past the key

12. Why is already sorted input $O(n)$?

- ☐ A. Each pass sorts the whole array
- ☐ B. Each pass immediately fails its first key comparison
- ☐ C. No outer iterations execute

▼ Answer key and explanations

1. **5.** Indexes start at 0, so the last index is $\text{length} - 1$.
2. **Extra storage stays bounded as n grows.** Auxiliary space counts extra working storage, excluding the input array.
3. **A statement that remains true at a specified loop boundary.** The invariant explains why each iteration preserves progress toward a correct result.
4. **The original relative order of records with equal keys.** Tags such as 2a and 2b let you check whether equal-key records crossed.
5. **Leave both unchanged.** Both inputs are already sorted; no key comparison is necessary.
6. **A shift may overwrite its original cell.** The separate variable preserves the value that must later fill the gap.
7. **Copy one value one position right.** $\text{arr}[j + 1] = \text{arr}[j]$ is one array write and temporarily duplicates a value.
8. **$\text{arr}[j + 1]$.** j is just left of the gap, either at a smaller/equal value or at -1 .
9. **Short-circuit evaluation prevents $\text{arr}[-1]$.** $\&\&$ stops evaluating when its left operand is false.
10. **3.** 85, 72 and 59 shift right. $45 > 51$ is false, so 51 is inserted after 45.
11. **Equal values are not shifted past the key.** Strictly larger values move; equals keep their original order.
12. **Each pass immediately fails its first key comparison.** There are $n - 1$ insertions but no shifts; each costs constant time.

The textbook definition

Sorting / Insertion sort · Definition

The textbook definition

Insertion sort is a comparison-based sorting algorithm that builds a sorted section of an array one element at a time. During each iteration, it takes the next element from the unsorted section, shifts all larger elements in the sorted section one position to the right, and inserts the

element into its correct position.

Comparison-based

The order is decided by comparing one value with another.

Sorted section

The part already placed in the correct order.

Unsorted section

The part that has not yet been processed.

Shift

Copying a value one position to the right.

Insert

Placing the saved value in the gap made by the shifts.

Iteration

One complete turn of the outer loop.

In plain language

Sorting / Insertion sort · In plain language

In plain language

Think about arranging playing cards in your hand. You pick up one new card, compare it with the cards already arranged, and slide larger cards aside until the new card fits.

Suppose the arranged cards are [2, 4, 5] and the new card is 3.

1 3 < 5

MOVE 5 RIGHT

2 3 < 4

MOVE 4 RIGHT

3 3 < 2

FALSE · STOP

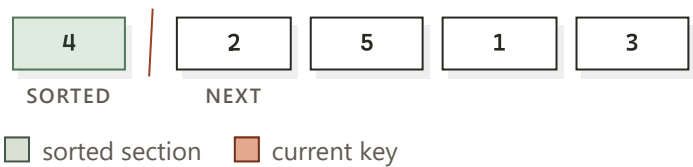
Place 3 after 2. The result is [2, 3, 4, 5].

The main idea

Sorting / Insertion sort · The main idea

The main idea

The first element is treated as sorted. That is valid because one item on its own cannot be out of order.



The algorithm processes 2, then 5, then 1, then 3. After each iteration, the sorted section grows by one value.

The repeating recipe: save the key → inspect values to its left → shift larger values right → insert the key.

Comparison, shift, insertion

Sorting / Insertion sort · Comparison, shift, insertion

Comparison, shift, insertion

These words describe different actions. Keeping them separate makes traces much easier to understand—and keeps test answers accurate.

Comparison Ask a question such as $1 < 5$.	Shift Copy a larger value one position right.
Insertion Put the saved key into the gap.	Swap Exchange two values. Standard insertion sort does not normally do this.

If 1 is the key in [2, 4, 5, 1, 3], then 5, 4, and 2 each shift right. Those are **three shifts**, not three swaps. The key is then inserted once.

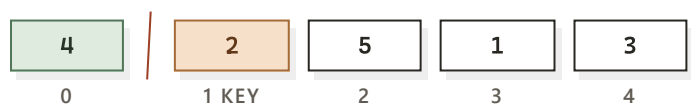
The complete visual trace

Sorting / Insertion sort · Visual trace

The complete visual trace

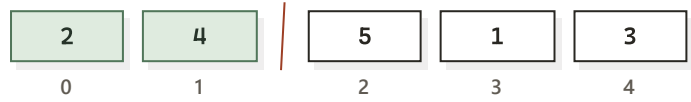
Iteration 1 · insert 2

1 comparison · 1 shift



1 2 < 4 **TRUE · SHIFT 4**

Save 2. Move 4 right, then insert 2 into the gap.



Iteration 2 · insert 5

1 comparison · 0 shifts



1 5 < 4 **FALSE · STOP**

5 is already larger than the value before it. Nothing moves. This is the **no-shift case**.



Iteration 3 · insert 1

3 comparisons · 3 shifts



1 1 < 5 **TRUE · SHIFT 5**

2 1 < 4 **TRUE · SHIFT 4**

3 1 < 2 **TRUE · SHIFT 2**

There is nothing else on the left. Insert 1 at index 0. This is the **multiple-shift case**.



Iteration 4 · insert 3

3 comparisons · 2 shifts



1 3 < 5 **TRUE · SHIFT 5**

2 3 < 4 **TRUE · SHIFT 4**

3 3 < 2 **FALSE · STOP**

Insert 3 after 2. The failed comparison is useful: it tells the algorithm where to stop.



The trace at a glance

Sorting / Insertion sort · Trace table

The trace at a glance

ITERATION	KEY	COMPARISONS	SHIFTS	ARRAY AFTER INSERTION
1	2	1	1	[2, 4, 5, 1, 3]
2	5	1	0	[2, 4, 5, 1, 3]
3	1	3	3	[1, 2, 4, 5, 3]
4	3	3	2	[1, 2, 3, 4, 5]
Total	—	8	6	Sorted

Comparisons and shifts are not always equal. In the last iteration, the third comparison is false: it causes no shift, but tells the algorithm to stop.

The Java implementation

Sorting / Insertion sort · Java implementation

The Java implementation

InsertionSort.java

```
public static void insertionSort(int[] array) {
    // Index 0 is already a sorted prefix of length one.
    for (int i = 1; i < array.length; i++) {
        // Save the next unsorted value before shifts overwrite its cell.
        int key = array[i];
        int j = i - 1;

        // Shift every larger prefix value one position right.
        while (j ≥ 0 && key < array[j]) {
            array[j + 1] = array[j];
            j--;
        }

        // j stopped before the key's correct insertion position.
        array[j + 1] = key;
    }
}
```

What each variable means

array The array being sorted.	i The index of the next unsorted value.
key A safe copy of the value being inserted.	j Moves backwards through the sorted section.

Why does **i** start at 1?

Index 0 already forms a one-value sorted section. The first value that needs inserting is therefore at index 1.

Why save the key?

Shifts can overwrite `array[i]`. Saving it in `key` prevents the original value from being lost.

Understanding the while condition

```
j ≥ 0 && key < array[j]
```

The algorithm may shift only when a valid position remains on the left *and* that value is larger than the key. The boundary check comes first so Java never attempts to access `array[-1]`.

How the shift and insertion happen

`array[j + 1] = array[j]` copies a larger value one place right. When the loop ends, `array[j + 1] = key` fills the gap with the saved key.

Write it from scratch

- 1 Start `i` at 1.
- 2 Save `key` and set `j = i - 1`.
- 3 Shift larger prefix values right.
- 4 Insert at `j + 1`.

Ascending ↔ descending

Ascending shifts while `key < array[j]`.

Descending shifts while `key > array[j]`.

Debug checklist

- Save the key before shifting.
- Check `j ≥ 0` first.
- Insert at `j + 1`, not `j`.

The same algorithm, expressed recursively

Sorting / Insertion sort · Recursive implementation

The same algorithm, expressed recursively

The recursive idea is: first sort the first `n - 1` values, then insert the final value into that sorted portion.

Base case

`n ≤ 1`. Zero or one item is already sorted.

Smaller problem

Sort only the first `n - 1` values.

Work after return

Insert `array[n - 1]` into the sorted prefix.

First call

`insertionSortRec(array, array.length)`.

Approach 1 · recursive outer process

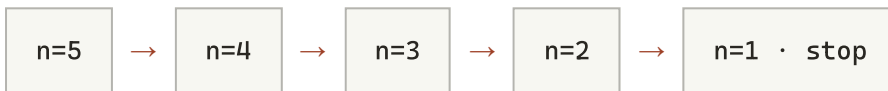
The recursion chooses how much of the array to sort. A loop still performs the insertion step.

Recursive sort + iterative insertion

```
public static void insertionSortRec(int[] array, int n) {  
    // Base case: a prefix of length zero or one is sorted.  
    if (n ≤ 1) return;  
  
    // First sort the smaller prefix.  
    insertionSortRec(array, n - 1);  
  
    // Then insert the final value into that sorted prefix.  
    int key = array[n - 1];  
    int j = n - 2;  
    while (j ≥ 0 && key < array[j]) {  
        array[j + 1] = array[j];  
        j--;  
    }  
    array[j + 1] = key;  
}
```

Call-and-return trace on [4, 2, 5, 1, 3]

CALLS BECOME SMALLER



RETURNS PERFORM THE INSERTIONS

- 1 Return to $n=1$. Base case: one value is already sorted, so nothing is inserted.
- 2 Return to $n=2$. Insert 2 into [4] → 1 shift; the array is now [2, 4, 5, 1, 3].
- 3 Return to $n=3$. Insert 5 into [2, 4] → no shift; the array is now [2, 4, 5, 1, 3].
- 4 Return to $n=4$. Insert 1 into [2, 4, 5] → 3 shifts; the array is now [1, 2, 4, 5, 3].
- 5 Return to $n=5$. Insert 3 into [1, 2, 4, 5] → 2 shifts; the array is now [1, 2, 3, 4, 5].

Approach 2 · fully recursive shifting

This version also replaces the insertion loop with a recursive helper. It is useful for understanding recursion, although the first version is usually simpler to read.

Recursive sort + recursive insertion

```
public static void insertionSortFullyRec(int[] a, int n) {
    // Stop shrinking at a one-value prefix.
    if (n ≤ 1) return;
    insertionSortFullyRec(a, n - 1);
    // Recursively place the last value into the sorted prefix.
    insertRec(a, a[n - 1], n - 2);
}

private static void insertRec(int[] a, int key, int j) {
    // Stop at the left edge or after a value no larger than key.
    if (j < 0 || a[j] ≤ key) {
        a[j + 1] = key;
        return;
    }
    a[j + 1] = a[j];
    insertRec(a, key, j - 1);
}
```

Trace · insertRec for the last step of [4, 2, 5, 1, 3]

The final return inserts 3 into the sorted prefix [1, 2, 4, 5]. The helper is called with $j = n - 2 = 3$.

CALL	WHAT HAPPENS	ARRAY AFTER
insertRec(a, 3, 3)	$a[3] = 5 > 3$: shift, $a[4] = 5$	[1, 2, 4, 5, 5]
insertRec(a, 3, 2)	$a[2] = 4 > 3$: shift, $a[3] = 4$	[1, 2, 4, 4, 5]
insertRec(a, 3, 1)	$a[1] = 2 \leq 3$: stop, so $a[2] = 3$	[1, 2, 3, 4, 5]

Complexity: the comparisons and shifts remain $O(n^2)$ in the worst case. Unlike the iterative version's $O(1)$ extra space, recursive calls require $O(n)$ extra call space.

Recursion, calls and returns

java

```
public static void insertionSortRec(int[] arr, int n) {
    // A prefix of zero or one value is already sorted.
    if (n ≤ 1) return;
    // Suspend this frame until the smaller prefix has been sorted.
    insertionSortRec(arr, n - 1);
    // Work happens ON RETURN: insert the final value into the sorted prefix.
    int key = arr[n - 1], j = n - 2;
    while (j ≥ 0 && arr[j] > key) {
        arr[j + 1] = arr[j];
        j--;
    }
    arr[j + 1] = key;
    // First call: insertionSortRec(arr, arr.length).
}
```

CALL STACK - NEWEST FRAME AT THE TOP

insertionSortRec(n=1) Base case returns immediately
insertionSortRec(n=2) Waiting: Insert 2 after child returns
insertionSortRec(n=3) Waiting: Insert 5 after child returns
insertionSortRec(n=4) Waiting: Insert 1 after child returns
insertionSortRec(n=5) Waiting: Insert 3 after child returns

Calls descend to $n=1$; insertions execute as $n=2,3,4,5$ return. The arrays after each return match the pass table.

A complete runnable Java example

Save as `InsertionDemo.java`. The code includes the method(s) and a main entry point.

java

```
import java.util.Arrays;

public class InsertionDemo {
    public static void insertionSort(int[] arr) {
        // Prefix 0..i-1 is already sorted before each insertion.
        for (int i = 1; i < arr.length; i++) {
            int key = arr[i]; // Preserve this value while its cell is overwritten.
            int j = i - 1;    // Compare backwards through the sorted prefix.
            // Check j first: && avoids evaluating arr[-1].
            // Strict > keeps equal values in their original order (stability).
            while (j ≥ 0 && arr[j] > key) {
                arr[j + 1] = arr[j]; // SHIFT: copy one value one cell right.
                j--;
            }
            // The gap is after the last value ≤ key, or at index 0.
            arr[j + 1] = key;
        }
    }

    public static void main(String[] args) {
        int[] values = {4, 2, 5, 1, 3};
        insertionSort(values);
        System.out.println(Arrays.toString(values));
    }
}
```

Expected console output

```
[1, 2, 3, 4, 5]
```

Properties

Sorting / Insertion sort · Properties

Properties

BEST TIME $O(n)$	AVERAGE TIME $O(n^2)$
WORST TIME $O(n^2)$	EXTRA SPACE $O(1)$
STABLE Yes	IN-PLACE Yes
ADAPTIVE Yes	TYPE Comparison

Stable

Equal values keep their original relative order. The condition uses $<$, so equal values are not moved past each other. Changing it to $\leq$ would remove that guarantee.

In-place

The values are rearranged inside the original array. Only a few variables are added, so extra space remains constant: $O(1)$.

Adaptive

Already sorted or nearly sorted data needs very few shifts. The algorithm notices this naturally; it does not keep scanning left once a comparison fails.

Comparison-based

Every placement decision comes from questions such as $3 < 5$ and $3 < 2$.

Why those complexities occur

Sorting / Insertion sort · Why those complexities occur

Why those complexities occur

Best case · $O(n)$

An already sorted array needs one failed comparison for each new key and no shifts.

Worst case · $O(n^2)$

A reverse-sorted array makes every new key cross the entire sorted section.

Best case

For [1, 2, 3, 4, 5], the comparisons $2 < 1$, $3 < 2$, $4 < 3$, and $5 < 4$ all fail immediately. That is $n - 1$ comparisons and no shifts, so the work grows directly with n .

Worst case

For [5, 4, 3, 2, 1], the new keys move past 1, then 2, then 3, then 4 earlier values.

$$1 + 2 + 3 + \dots + (n - 1) = n(n - 1) / 2 \rightarrow O(n^2)$$

Average case

In random data, a key moves across roughly half of the sorted section on average. The constant is smaller than in the worst case, but the dominant growth is still proportional to n^2 .

When should you use it?

Sorting / Insertion sort · When to use it

When should you use it?

A good choice

Small arrays; nearly sorted values; data arriving one item at a time; very limited memory; cases that need stability.

A poor choice

Very large arrays; reverse-ordered data; situations requiring predictable $O(n \log n)$ performance.

Insertion sort is excellent for a small hand of cards or adding a new score to an already sorted leaderboard. For a large, disorderly array, merge sort is usually more efficient.

Why sort data at all?

Sorting · Why sort data at all?

Why sort data at all?

Before the algorithms: what does sorted data give you that unsorted data does not, and what does it cost?

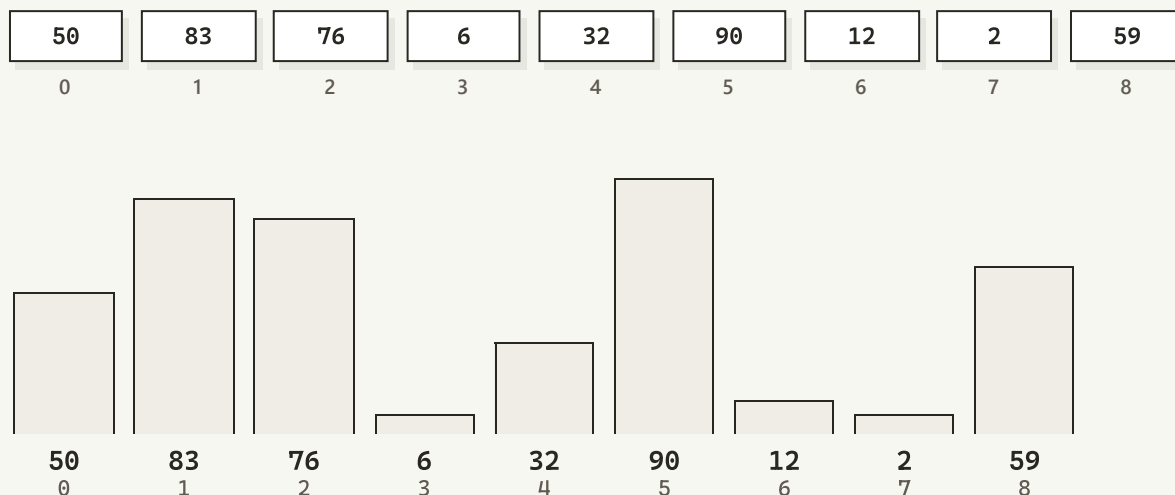
Everything in this chapter is about putting values in order. Before learning *how*, it is worth being clear about **why**, because it is a favourite exam question and because it explains why searching and sorting are taught together.

Short answer for a test: sorting data makes it easier and faster to find information in it. A sorted array can be searched with binary search, in $O(\log n)$ instead of $O(n)$.

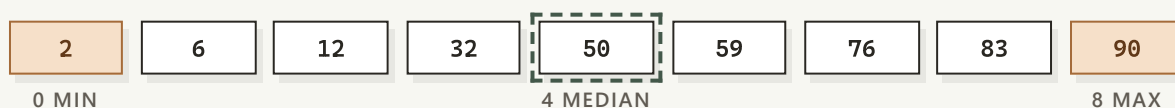
What becomes easy once data is sorted

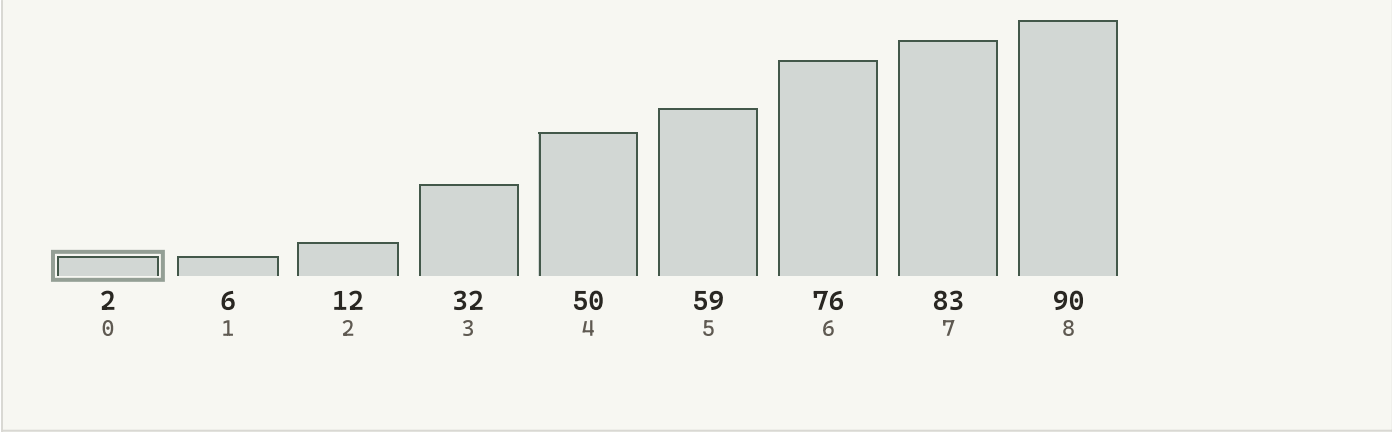
Take the nine values from the past paper. Unsorted they tell you nothing at a glance; sorted, a lot of questions answer themselves.

UNSORTED



THE SAME VALUES, SORTED ASCENDING





QUESTION	UNSORTED ARRAY	SORTED ARRAY
Smallest and largest value?	Look at all 9 cells	The first and last cell: 2 and 90
Median (middle) value?	Needs a lot of work	The middle cell: 50
Is a value present?	Linear search: up to 9 comparisons	Binary search: at most 4 probes
Where are the duplicates?	Anywhere	Next to each other (see below)
Merge two lists?	No shortcut	Merge them by zipping (that is merge sort's merge step)
Show a list to a person?	Hard to read	Alphabetical or numerical order, easy to scan

Duplicates end up side by side

Finding repeated values in [3, 1, 3, 2, 3] means comparing every pair. After sorting, [1, 2, 3, 3, 3], equal values are neighbours, so one pass comparing each cell with the next finds them all.

Sorting is not free: is it worth it?

Sorting costs about $n \log_2 n$ steps, and it only pays off if you search more than a few times. Suppose you have **1,000,000** values and will search for **1,000** different keys:

STRATEGY	WORK	TOTAL
Never sort: linear search every time	1,000 searches × up to 1,000,000 comparisons	1,000,000,000
Sort once, then binary search every time	sort ≈ 20,000,000 steps, then 1,000 searches × 20 probes	20,020,000

That is about **49× less work** after sorting. With a single search the answer flips: scanning once costs 1,000,000 and sorting first costs 20,000,000. The rule: *search once, scan; search often, sort first.*

▼ Quick check: Exam-style question 3(a), “What is the purpose of sorting data?”

To arrange data in a defined order (ascending or descending) so that information is easier and faster to find, for example by using binary search instead of scanning every element. Other valid points: duplicates become adjacent, minimum and maximum are at the ends, and sorted data is easier to read and merge.

The sorting map

Sorting

The sorting map

Every algorithm orders values, but each uses a different repeated action. The four are taught in this order: Bubble, Selection, Insertion, Merge.

ALGORITHM	REPEATED ACTION	TYPICAL TIME	BEST FIT
Bubble	Compare neighbours; swap wrong pairs	$O(n^2)$	Learning and tiny inputs
Selection	Find the smallest remaining value; place it next	$O(n^2)$	When few swaps matter
Insertion	Shift larger sorted values; insert a key	$O(n^2)$	Small or nearly sorted data
Merge	Divide into halves; merge sorted halves	$O(n \log n)$	Large data and predictable speed

Vocabulary behind every property tile

Comparison-based Order is decided only by asking questions such as $a < b$.	Best / average / worst The same algorithm on three arrangements of the same n values: already ordered, typical, and the most awkward.
In-place Values are rearranged inside the original array using only a few extra variables: $O(1)$ auxiliary space.	Auxiliary space Extra memory beyond the input array, such as the temporary copies merge sort makes.
Adaptive Does less work when the input is already or nearly sorted.	Stable Equal values keep their original relative order.

Why stability matters

Sort these students by mark. Ana and Ben both scored 70, and Ana was listed first.

Start

70	70	65
ANA	BEN	CY

Stable sort result

65	70	70
CY	ANA	BEN

An unstable sort may give

65	70	70
CY	BEN	ANA

Both outputs are sorted by mark. Only the stable one keeps Ana before Ben, which matters when you sort by one field and want an earlier ordering by another field preserved.

Count the disorder: inversions

An **inversion** is a pair of positions whose values are in the wrong order: a larger value sits to the left of a smaller one. $[4, 2, 5, 1, 3]$ has six: $(4,2)$, $(4,1)$, $(4,3)$, $(2,1)$, $(5,1)$, $(5,3)$.

ALGORITHM	WHAT IS COUNTED	ON $[4, 2, 5, 1, 3]$	LINK TO INVERSIONS
Bubble	adjacent swaps	6	Each swap of neighbours removes exactly one inversion.
Insertion	shifts	6	Each shift removes exactly one inversion.
Selection	swaps	2	At most $n - 1$ swaps; one swap can remove several inversions at once.

A fully reversed array of n values has $n(n - 1) / 2$ inversions. For $[5, 4, 3, 2, 1]$ that is 10, which equals the 10 shifts in the reverse-order insertion example. This is the reason bubble and insertion sort are $O(n^2)$ in the worst case.

Reverse the order by reversing one comparison

Sorting / Insertion sort · Descending order

Reverse the order by reversing one comparison

Only the shift condition changes. Everything else, including saving the key, starting at index 1, and inserting at $j + 1$, stays identical.

Two versions of the while line

```
// Ascending: shift while the key is SMALLER than the value on its left.
while (j ≥ 0 && key < array[j]) {
    array[j + 1] = array[j];
    j--;
}

// Descending: shift while the key is LARGER than the value on its left.
while (j ≥ 0 && key > array[j]) {
    array[j + 1] = array[j];
    j--;
}
```

The same input in both directions

Each cell reads comparisons / shifts for $[4, 2, 5, 1, 3]$.

KEY	ASCENDING C / S	ARRAY AFTER (ASCENDING)	DESCENDING C / S	ARRAY AFTER (DESCENDING)
2	1 / 1	[2, 4, 5, 1, 3]	1 / 0	[4, 2, 5, 1, 3]
5	1 / 0	[2, 4, 5, 1, 3]	2 / 2	[5, 4, 2, 1, 3]
1	3 / 3	[1, 2, 4, 5, 3]	1 / 0	[5, 4, 2, 1, 3]
3	3 / 2	[1, 2, 3, 4, 5]	3 / 2	[5, 4, 3, 2, 1]
Total	8 / 6	[1, 2, 3, 4, 5]	7 / 4	[5, 4, 3, 2, 1]

Ascending: 8 comparisons, 6 shifts. Descending: 7 comparisons, 4 shifts.

The costs differ because “out of order” depends on the direction. This array has 10 pairs of positions. Six have a larger value on the left, and those cost a shift when sorting ascending. The other four have a smaller value on the left, and those cost a shift when sorting descending: $6 + 4 = 10$.

Stability is unchanged. Descending still uses a strict $>$, so equal values do not shift past each other.

▼ **Quick check: for an already *descending* array, which direction is the best case?**

Sorting descending. Every key is already not larger than the value on its left, so each iteration makes one failed comparison and no shifts: $O(n)$.

Exam-style question 3(b): descending trace

Sorting / Insertion sort · Exam-style question 3(b): descending trace

Exam-style question 3(b): descending trace

The insertion-sort question from the past paper, solved step by step.

This is the past paper, question 3(b): “Show the contents of the array below at the end of each pass when sorting the array using the insertion sort algorithm, assuming **descending** order.” The array is [50, 83, 76, 6, 32, 90, 12, 2, 59].

Before you start: what changes for descending?

In ascending insertion sort the key moves left while it is *smaller* than the value before it. In descending order the key moves left while it is *larger*. In code, the only change is the comparison in the `while` loop:

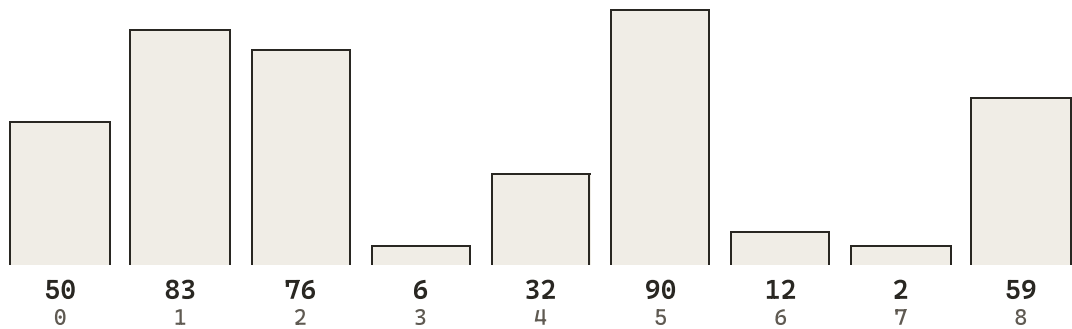
Ascending

```
while (j ≥ 0 && key < arr[j]) {  
    arr[j + 1] = arr[j];  
    j--;  
}
```

Descending (this question)

```
while (j ≥ 0 && key > arr[j]) {  
    arr[j + 1] = arr[j];  
    j--;  
}
```

Use the descending rule on every row below: **the largest values end up on the left**, so the finished array is [90, 83, 76, 59, 50, 32, 12, 6, 2]. A pass is one value of `i` (the outer loop); there are 8 passes for nine values.



The answer, pass by pass

PASS	ARRAY AT THE END OF THE PASS	OPERATION
Start	[50, 83, 76, 6, 32, 90, 12, 2, 59]	(unsorted: only the first value is a sorted section of length 1)
1	[83, 50, 76, 6, 32, 90, 12, 2, 59]	key = 83: 83 > 50 true → shift 50 right; front of the array reached → stop; insert 83 at index 0
2	[83, 76, 50, 6, 32, 90, 12, 2, 59]	key = 76: 76 > 50 true → shift 50 right; 76 > 83 false → stop; insert 76 at index 1
3	[83, 76, 50, 6, 32, 90, 12, 2, 59]	key = 6: 6 > 50 false → stop; insert 6 at index 3 (no shift needed)
4	[83, 76, 50, 32, 6, 90, 12, 2, 59]	key = 32: 32 > 6 true → shift 6 right; 32 > 50 false → stop; insert 32 at index 3
5	[90, 83, 76, 50, 32, 6, 12, 2, 59]	key = 90: 90 > 6 true → shift 6 right; 90 > 32 true → shift 32 right; 90 > 50 true → shift 50 right; 90 > 76 true → shift 76 right; 90 > 83 true → shift 83 right; front of the array reached → stop; insert 90 at index 0
6	[90, 83, 76, 50, 32, 12, 6, 2, 59]	key = 12: 12 > 6 true → shift 6 right; 12 > 32 false → stop; insert 12 at index 5
7	[90, 83, 76, 50, 32, 12, 6, 2, 59]	key = 2: 2 > 6 false → stop; insert 2 at index 7 (no shift needed)
8	[90, 83, 76, 59, 50, 32, 12, 6, 2]	key = 59: 59 > 2 true → shift 2 right; 59 > 6 true → shift 6 right; 59 > 12 true → shift 12 right; 59 > 32 true → shift 32 right; 59 > 50 true → shift 50 right; 59 > 76 false → stop; insert 59 at index 3

Totals: 20 comparisons and 14 shifts. In a test you only need the middle column, but writing the operation (as above) shows the examiner you followed the algorithm and earns method marks even if one row slips.

The same trace as pictures

The divider separates the sorted section (left) from the unsorted rest. The darker cell is the key that was just inserted.

Start

50	83	76	6	32	90	12	2	59
0	1	2	3	4	5	6	7	8

Pass 1 · key = 83 · 1 shift

83	50	76	6	32	90	12	2	59
0	1	2	3	4	5	6	7	8

Pass 2 · key = 76 · 1 shift

83	76	50	6	32	90	12	2	59
0	1	2	3	4	5	6	7	8

Pass 3 · key = 6 · 0 shifts

83	76	50	6	32	90	12	2	59
0	1	2	3	4	5	6	7	8

Pass 4 · key = 32 · 1 shift

83	76	50	32	6	90	12	2	59
0	1	2	3	4	5	6	7	8

Pass 5 · key = 90 · 5 shifts

90	83	76	50	32	6	12	2	59
0	1	2	3	4	5	6	7	8

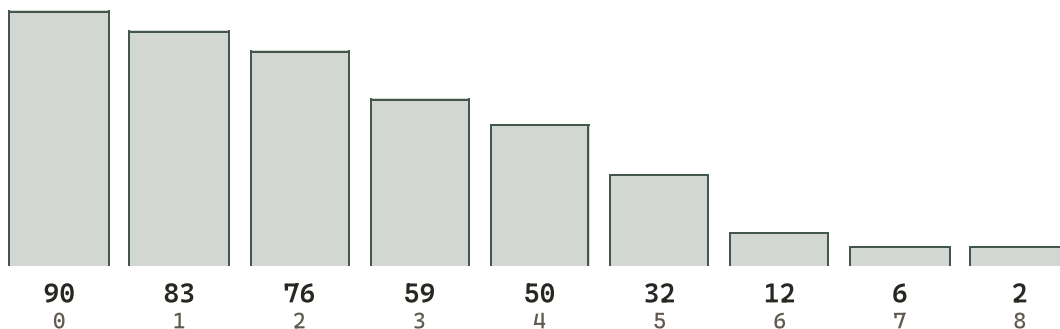
Pass 6 · key = 12 · 1 shift

90	83	76	50	32	12	6	2	59
0	1	2	3	4	5	6	7	8

Pass 7 · key = 2 · 0 shifts



Pass 8 · key = 59 · 5 shifts



Descending order drawn as bars: a staircase going *down*.

How to check your own answer: after every pass, the first $i + 1$ values must be in descending order, and together they must be exactly the first $i + 1$ original values. The cells to the right of the divider must be untouched.

Four patterns you must recognise

Sorting / Insertion sort · Worked examples

Four patterns you must recognise

Already sorted · [1, 2, 3, 4, 5]

Every key is already larger than the value before it.

Start · the first value is already a sorted section of length 1



Insert 2 · $i = 1$

$2 < 1$ false → stop. Insert 2 at index 1.



Insert 3 · $i = 2$

$3 < 2$ false → stop. Insert 3 at index 2.



Insert 4 · $i = 3$

$4 < 3$ false → stop. Insert 4 at index 3.



Insert 5 · $i = 4$

$5 < 4$ false → stop. Insert 5 at index 4.



KEY	COMPARISONS	SHIFTS	ARRAY AFTER INSERTION
2	1	0	[1, 2, 3, 4, 5]
3	1	0	[1, 2, 3, 4, 5]
4	1	0	[1, 2, 3, 4, 5]
5	1	0	[1, 2, 3, 4, 5]

KEY	COMPARISONS	SHIFTS	ARRAY AFTER INSERTION
Total	4	0	Sorted

4 comparisons, 0 shifts: $n - 1$ comparisons, the $O(n)$ best case.

Reverse order · [5, 4, 3, 2, 1]

Every key is smaller than everything in the sorted section, so it crosses all of it.

Start · the first value is already a sorted section of length 1



Insert 4 · $i = 1$

$4 < 5$ true → shift 5; no cell remains on the left → stop. Insert 4 at index 0.



Insert 3 · $i = 2$

$3 < 5$ true → shift 5; $3 < 4$ true → shift 4; no cell remains on the left → stop. Insert 3 at index 0.



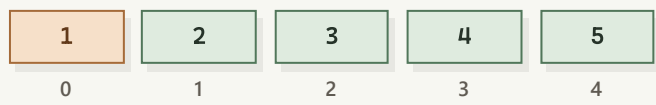
Insert 2 · $i = 3$

$2 < 5$ true → shift 5; $2 < 4$ true → shift 4; $2 < 3$ true → shift 3; no cell remains on the left → stop. Insert 2 at index 0.



Insert 1 · i = 4

1 < 5 true → shift 5; 1 < 4 true → shift 4; 1 < 3 true → shift 3; 1 < 2 true → shift 2; no cell remains on the left → stop. Insert 1 at index 0.



KEY	COMPARISONS	SHIFTS	ARRAY AFTER INSERTION
4	1	1	[4, 5, 3, 2, 1]
3	2	2	[3, 4, 5, 2, 1]
2	3	3	[2, 3, 4, 5, 1]
1	4	4	[1, 2, 3, 4, 5]
Total	10	10	Sorted

10 comparisons and 10 shifts: $1 + 2 + 3 + 4 = n(n - 1) / 2$, the $O(n^2)$ worst case.

Nearly sorted · [1, 2, 4, 3, 5]

Only one pair is out of order, so only one key moves.

Start · the first value is already a sorted section of length 1



Insert 2 · i = 1

2 < 1 false → stop. Insert 2 at index 1.



Insert 4 · i = 2

4 < 2 false → stop. Insert 4 at index 2.



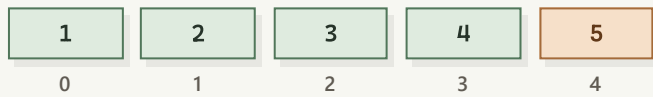
Insert 3 · i = 3

3 < 4 true → shift 4; 3 < 2 false → stop. Insert 3 at index 2.



Insert 5 · i = 4

5 < 4 false → stop. Insert 5 at index 4.



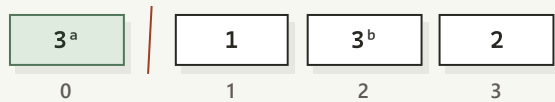
KEY	COMPARISONS	SHIFTS	ARRAY AFTER INSERTION
2	1	0	[1, 2, 4, 3, 5]
4	1	0	[1, 2, 4, 3, 5]
3	2	1	[1, 2, 3, 4, 5]
5	1	0	[1, 2, 3, 4, 5]
Total	5	1	Sorted

5 comparisons and 1 shift. The cost tracks how disordered the input is, which is what **adaptive** means.

Duplicates · [3^a, 1, 3^b, 2]

Two equal 3s are labelled ^a and ^b so their order can be tracked.

Start · the first value is already a sorted section of length 1



Insert $1 \cdot i = 1$

$1 < 3^a$ true $\rightarrow$ shift 3^a ; no cell remains on the left $\rightarrow$ stop. Insert 1 at index 0.



Insert $3^b \cdot i = 2$

$3^b < 3^a$ false $\rightarrow$ stop. Insert 3^b at index 2.



Insert $2 \cdot i = 3$

$2 < 3^b$ true $\rightarrow$ shift 3^b ; $2 < 3^a$ true $\rightarrow$ shift 3^a ; $2 < 1$ false $\rightarrow$ stop. Insert 2 at index 1.



KEY	COMPARISONS	SHIFTS	ARRAY AFTER INSERTION
1	1	1	[1, 3^a , 3^b , 2]
3^b	1	0	[1, 3^a , 3^b , 2]
2	3	2	[1, 2, 3^a , 3^b]
Total	5	3	Sorted

5 comparisons, 3 shifts. $3^b < 3^a$ is false, so 3^a stays ahead of 3^b . Equal values never cross, which is what **stable** means.

Questions

▼ Why is shifting preferred to swapping the key repeatedly?

Saving one key and shifting a block makes the algorithm's purpose clearer and avoids unnecessary repeated exchanges.

▼ What happens if the while comparison uses $\leq$?

Equal values may move past one another, so the implementation can lose stability.

▼ Does descending order cost the same as ascending order?

Not necessarily. On `[4, 2, 5, 1, 3]` ascending makes 8 comparisons and 6 shifts, while descending makes 7 comparisons and 4 shifts. Each shift removes one out-of-order pair, and the number of such pairs depends on the direction. See [Descending order](#).

Insertion sort in one sentence

Take the next key, shift larger values in the sorted prefix right, and place the key into the gap.

Common mistakes

Sorting / Insertion sort · Common mistakes

Common mistakes

- 1 Calling shifts swaps.**
Insertion sort saves the key, shifts larger values, and inserts the key. It does not normally exchange pairs.
- 2 Starting at index 0.**
The first element already forms the initial sorted section, so the outer loop begins at index 1.
- 3 Forgetting to save the key.**
A shift may overwrite its array position. Save `array[i]` before moving anything.

- 4 **Checking the value before the boundary.**
Use `j ≥ 0 && key < array[j]`. The reverse order may attempt to read index `-1`.
- 5 **Using $\leq$ unnecessarily.**
Moving equal values can destroy the algorithm's stability. Use `<`.
- 6 **Forgetting the final insertion.**
After shifting, place the key with `array[j + 1] = key`.

A test-ready summary

Sorting / Insertion sort · Test summary

A test-ready summary

TEST ANSWER

Insertion sort is a stable, in-place and adaptive comparison-based sorting algorithm. It builds a sorted section from left to right. During each iteration, the next value is saved as the key, larger values in the sorted section are shifted to the right, and the key is inserted into the resulting gap. Its best-case time complexity is $O(n)$ for an already sorted array, while its average and worst-case complexities are $O(n^2)$. It is suitable for small or nearly sorted arrays but inefficient for large, randomly ordered arrays.

Practice

Sorting / Insertion sort · Practice

Practice

Guided question

Use insertion sort to arrange [6, 3, 8, 4]. Record the key, comparisons, shifts, and array after each iteration.

▼ **Reveal the guided answer**

Insert 3: 3 < 6, so shift 6 → [3, 6, 8, 4].

Insert 8: 8 < 6 is false, so nothing moves.

Insert 4: shift 8, shift 6, then stop at 3 → [3, 4, 6, 8].

Totals: 5 comparisons and 3 shifts.

Independent question

Trace insertion sort on [7, 2, 5, 1, 6].

- ☐ Write the key for every iteration.
- ☐ Record every comparison.
- ☐ Record every shift.
- ☐ Write the resulting array after each insertion.
- ☐ Calculate the totals.

▼ **Reveal the independent answer**

KEY	COMPARISONS	SHIFTS	ARRAY AFTER INSERTION
2	1	1	[2, 7, 5, 1, 6]
5	2	1	[2, 5, 7, 1, 6]
1	3	3	[1, 2, 5, 7, 6]
6	2	1	[1, 2, 5, 6, 7]
Total	8	6	Sorted

Key 1 crosses the entire sorted section (3 shifts) and reaches the left edge, so it makes no fourth comparison.

More worked cases: edge cases, duplicates and descending order

Each case has input and result diagrams, evaluated decisions or merge inputs, exact counts, and a final state.

1. Nearly sorted

1

2

4

3

5

01234

↓ ascending insertion sort

1

2

3

4

5

01234

The trace below connects the input diagram to the final result.

PASS	BEFORE	EVALUATED CONDITIONS	AFTER	KEY COMPARISONS
1	[1,2,4,3,5]	1 > 2 → false	[1,2,4,3,5]	1
2	[1,2,4,3,5]	2 > 4 → false	[1,2,4,3,5]	1
3	[1,2,4,3,5]	4 > 3 → true; 2 > 3 → false	[1,2,3,4,5]	2
4	[1,2,3,4,5]	4 > 5 → false	[1,2,3,4,5]	1

5 key comparisons; 1 shifts.

2. Reverse order

5

4

3

2

1

01234

↓ ascending insertion sort

1

2

3

4

5

01234

The trace below connects the input diagram to the final result.

PASS	BEFORE	EVALUATED CONDITIONS	AFTER	KEY COMPARISONS
1	[5,4,3,2,1]	5 > 4 → true	[4,5,3,2,1]	1
2	[4,5,3,2,1]	5 > 3 → true; 4 > 3 → true	[3,4,5,2,1]	2
3	[3,4,5,2,1]	5 > 2 → true; 4 > 2 → true; 3 > 2 → true	[2,3,4,5,1]	3
4	[2,3,4,5,1]	5 > 1 → true; 4 > 1 → true; 3 > 1 → true; 2 > 1 → true	[1,2,3,4,5]	4

10 key comparisons; 10 shifts.

3. Duplicate values

2

2

1

3

0123

↓ ascending insertion sort

1

2

2

3

0123

The trace below connects the input diagram to the final result.

PASS	BEFORE	EVALUATED CONDITIONS	AFTER	KEY COMPARISONS
1	[2,2,1,3]	$2 > 2 \rightarrow \text{false}$	[2,2,1,3]	1
2	[2,2,1,3]	$2 > 1 \rightarrow \text{true}; 2 > 1 \rightarrow \text{true}$	[1,2,2,3]	2
3	[1,2,2,3]	$2 > 3 \rightarrow \text{false}$	[1,2,2,3]	1

4 key comparisons; 2 shifts.

4. Uneven length

38

27

43

3

9

01234

↓ ascending insertion sort

3

9

27

38

43

01234

The trace below connects the input diagram to the final result.

PASS	BEFORE	EVALUATED CONDITIONS	AFTER	KEY COMPARISONS
1	[38,27,43,3,9]	$38 > 27 \rightarrow \text{true}$	[27,38,43,3,9]	1
2	[27,38,43,3,9]	$38 > 43 \rightarrow \text{false}$	[27,38,43,3,9]	1
3	[27,38,43,3,9]	$43 > 3 \rightarrow \text{true}; 38 > 3 \rightarrow \text{true}; 27 > 3 \rightarrow \text{true}$	[3,27,38,43,9]	3
4	[3,27,38,43,9]	$43 > 9 \rightarrow \text{true}; 38 > 9 \rightarrow \text{true}; 27 > 9 \rightarrow \text{true}; 3 > 9 \rightarrow \text{false}$	[3,9,27,38,43]	4

9 key comparisons; 7 shifts.

5. Descending target

8

3

5

1

0123

↓ descending insertion sort

8

5

3

1

0123

The trace below connects the input diagram to the final result.

PASS	BEFORE	EVALUATED CONDITIONS	AFTER	KEY COMPARISONS
1	[8,3,5,1]	$8 < 3 \rightarrow \text{false}$	[8,3,5,1]	1
2	[8,3,5,1]	$3 < 5 \rightarrow \text{true}; 8 < 5 \rightarrow \text{false}$	[8,5,3,1]	2
3	[8,5,3,1]	$3 < 1 \rightarrow \text{false}$	[8,5,3,1]	1

4 key comparisons; 1 shifts. Change `arr[j] > key` to `arr[j] < key`. Shift smaller prefix values right, stopping after a value at least as large as the key.

Exam-style practice with worked answers

▼ A. Trace [8, 3, 5, 1] in descending order. Show every completed pass/merge. [8]

PASS	BEFORE	AFTER	KEY COMPARISONS
1	[8,3,5,1]	[8,3,5,1]	1
2	[8,3,5,1]	[8,5,3,1]	2
3	[8,5,3,1]	[8,5,3,1]	1

Final result [8,5,3,1]. Change $\text{arr}[j] > \text{key}$ to $\text{arr}[j] < \text{key}$. Shift smaller prefix values right, stopping after a value at least as large as the key.

▼ B. Explain stability with tagged equal keys. [4]

Strict inversion comparisons do not move a value past an equal one. Using $\geq$ (or $\leq$ descending) would remove that guarantee.

▼ C. Give the time and space, then justify both. [5]

Best: $O(n)$. Already sorted input makes one failed key comparison per insertion. Average / worst: $O(n^2)$. Reverse input shifts across every earlier prefix cell. Auxiliary space: $O(1)$ iterative; $O(n)$ recursive. One saved key versus suspended recursive frames. Stable / in place / adaptive: Yes / yes / yes. Strict $>$ never shifts an equal value past the key; few inversions mean few shifts.

▼ D. Debug the code before tracing it. [4]

MISTAKE	SYMPTOM	FIX
Overwrite arr[i] without saving it	The key is lost during shifting.	Save int key = arr[i] first.
Evaluate arr[j] before j >= 0	May read arr[-1] at the left edge.	Place the bounds condition first with &&.

▼ E. What happens for [] and [7]? [2]

Both are already sorted. No key comparisons or value movements are required. Merge sort returns at its base case; the iterative methods never enter a pass.